

Homework: The EM Algorithm

Problem 1: EM for a Two-Component Exponential Mixture

Suppose you observe n data points $y_1, \dots, y_n$ from a two-component exponential mixture:

$$f(y_i|\theta) = \pi \lambda_1 e^{-\lambda_1 y_i} + (1 - \pi) \lambda_2 e^{-\lambda_2 y_i}, \quad y_i > 0$$

where $\theta = (\pi, \lambda_1, \lambda_2)$.

(a) Write down the complete-data log-likelihood, treating the component membership $z_i \in \{1, 2\}$ as the latent variable.

Let z_{i1} and z_{i2} be latent indicator variables where $z_{i1} = 1$ if y_i is from component 1, and $z_{i2} = 1$ if from component 2 ($z_{i1} + z_{i2} = 1$).

The complete-data log-likelihood is:

$$\ell^c(\pi, \lambda_1, \lambda_2) = \sum_{i=1}^n [z_{i1}(\ln \pi + \ln \lambda_1 - \lambda_1 y_i) + z_{i2}(\ln(1 - \pi) + \ln \lambda_2 - \lambda_2 y_i)]$$

(b) Derive the E-step: compute the responsibility $\gamma_i^{(t)} = P(z_i = 1 | y_i, \theta^{(t)})$.

By Bayes' theorem, the responsibility $\gamma_i^{(t)}$ is the posterior probability that observation y_i belongs to component 1, given the current parameter estimates $\theta^{(t)} = (\pi^{(t)}, \lambda_1^{(t)}, \lambda_2^{(t)})$:

$$\gamma_i^{(t)} = \frac{P(y_i | z_i = 1, \theta^{(t)}) P(z_i = 1 | \theta^{(t)})}{P(y_i | \theta^{(t)})}$$

Substituting the prior probabilities and the exponential probability density functions into the numerator and the marginal likelihood into the denominator, we get:

$$\gamma_i^{(t)} = \frac{\pi^{(t)} \lambda_1^{(t)} e^{-\lambda_1^{(t)} y_i}}{\pi^{(t)} \lambda_1^{(t)} e^{-\lambda_1^{(t)} y_i} + (1 - \pi^{(t)}) \lambda_2^{(t)} e^{-\lambda_2^{(t)} y_i}}$$

(c) Derive the M-step: write the update formulas for $\pi^{(t+1)}$, $\lambda_1^{(t+1)}$, and $\lambda_2^{(t+1)}$.

To perform the M-step, we maximize the expected complete-data log-likelihood (the Q -function) with respect to each parameter. We replace the latent variable z_{i1} with the responsibility $\gamma_i^{(t)}$ and z_{i2} with $(1 - \gamma_i^{(t)})$.

The Q -function is:

$$Q(\theta | \theta^{(t)}) = \sum_{i=1}^n [\gamma_i^{(t)}(\ln \pi + \ln \lambda_1 - \lambda_1 y_i) + (1 - \gamma_i^{(t)})(\ln(1 - \pi) + \ln \lambda_2 - \lambda_2 y_i)]$$

1. Update for $\pi^{(t+1)}$: Take the derivative of Q with respect to π and set it to zero:

$$\frac{\partial Q}{\partial \pi} = \sum_{i=1}^n \left(\frac{\gamma_i^{(t)}}{\pi} - \frac{1 - \gamma_i^{(t)}}{1 - \pi} \right) = 0$$

Solving for π yields the average responsibility:

$$\pi^{(t+1)} = \frac{1}{n} \sum_{i=1}^n \gamma_i^{(t)}$$

2. Update for $\lambda_1^{(t+1)}$: Take the derivative of Q with respect to λ_1 and set it to zero:

$$\frac{\partial Q}{\partial \lambda_1} = \sum_{i=1}^n \gamma_i^{(t)} \left(\frac{1}{\lambda_1} - y_i \right) = 0$$

Solving for λ_1 yields:

$$\lambda_1^{(t+1)} = \frac{\sum_{i=1}^n \gamma_i^{(t)}}{\sum_{i=1}^n \gamma_i^{(t)} y_i}$$

3. Update for $\lambda_2^{(t+1)}$: Similarly, take the derivative of Q with respect to λ_2 and set it to zero:

$$\frac{\partial Q}{\partial \lambda_2} = \sum_{i=1}^n (1 - \gamma_i^{(t)}) \left(\frac{1}{\lambda_2} - y_i \right) = 0$$

Solving for λ_2 yields:

$$\lambda_2^{(t+1)} = \frac{\sum_{i=1}^n (1 - \gamma_i^{(t)})}{\sum_{i=1}^n (1 - \gamma_i^{(t)}) y_i}$$

(d) Implement the EM algorithm for this model. Your function should have the following signature and return a dictionary containing the estimated parameters and the log-likelihood history.

```
import numpy as np

def em_exponential_mixture(y, pi_init, lam1_init, lam2_init,
                           max_iter=200, tol=1e-8):
    """EM algorithm for a 2-component exponential mixture.

    Parameters
    -----
    y : np.ndarray
        Observed data of shape (n,), all positive.
    pi_init : float
        Initial mixing proportion for component 1.
    lam1_init, lam2_init : float
        Initial rate parameters.
    max_iter : int
        Maximum number of iterations.
    tol : float
        Convergence tolerance on log-likelihood change.

    Returns
    -----
    dict with keys: pi, lam1, lam2, loglik_history
    """
    pi = pi_init
    lam1 = lam1_init
    lam2 = lam2_init
    loglik_history = []
    n = len(y)

    for _ in range(max_iter):
        # E-step prerequisites: Calculate pdfs and marginal likelihood
        pdf1 = lam1 * np.exp(-lam1 * y)
        pdf2 = lam2 * np.exp(-lam2 * y)
        marginal = pi * pdf1 + (1 - pi) * pdf2

        # Log-likelihood
        loglik = np.sum(np.log(marginal))

        # Check convergence
        if len(loglik_history) > 0 and abs(loglik - loglik_history[-1]) < tol:
            loglik_history.append(loglik)
            break
        loglik_history.append(loglik)

        # E-step: Responsibilities
        gamma = (pi * pdf1) / marginal

        # M-step: Parameter updates
        sum_gamma = np.sum(gamma)
        sum_1_minus_gamma = n - sum_gamma

        pi = sum_gamma / n
        lam1 = sum_gamma / np.sum(gamma * y)
        lam2 = sum_1_minus_gamma / np.sum((1 - gamma) * y)

    return {
        'pi': pi,
        'lam1': lam1,
        'lam2': lam2,
        'loglik_history': loglik_history
    }
```

Test your implementation on the following simulated data and report the estimated parameters:

```
np.random.seed(123)
n = 500
pi_true = 0.35
lam1_true, lam2_true = 0.5, 3.0
z = np.random.binomial(1, 1 - pi_true, n)
y = np.where(z == 0,
             np.random.exponential(1 / lam1_true, n),
             np.random.exponential(1 / lam2_true, n))
```

```
result = em_exponential_mixture(y, pi_init=0.5, lam1_init=0.3, lam2_init=2.0)
print(result)
```

```
{'pi': np.float64(0.2992140129215983), 'lam1': np.float64(0.4353765194014798), 'lam2': np.float64(2.4792521253782076)}
```

✓ Problem 2: Diagnosing EM Convergence

A colleague provides the following EM implementation for the two-component Gaussian mixture model. Read the code carefully and answer the questions below.

```
import numpy as np
from scipy import stats

def em_gmm_v2(y, pi_init, mu1_init, sigma1_init, mu2_init, sigma2_init,
              max_iter=200, tol=1e-8):
    n = len(y)
    pi = pi_init
    mu1, mu2 = mu1_init, mu2_init
    s1, s2 = sigma1_init, sigma2_init
    loglik_history = []

    for iteration in range(max_iter):
        d1 = pi * stats.norm.pdf(y, mu1, s1)
        d2 = (1 - pi) * stats.norm.pdf(y, mu2, s2)
        gamma = d1 / (d1 + d2)

        ll = np.sum(np.log(d1 + d2))
        loglik_history.append(ll)

        if iteration > 0 and loglik_history[-1] - loglik_history[-2] < tol:
            break

        n1 = np.sum(gamma)
        n2 = n - n1

        pi = n1 / n
        mu1 = np.sum(gamma * y) / n1
        mu2 = np.sum((1 - gamma) * y) / n2
        s1 = np.sqrt(np.sum(gamma * (y - mu1) ** 2) / n1)
        s2 = np.sqrt(np.sum((1 - gamma) * (y - mu2) ** 2) / n2)

    return {
        "pi": pi, "mu1": mu1, "sigma1": s1,
        "mu2": mu2, "sigma2": s2,
        "loglik_history": loglik_history,
    }
```

(a) There is a subtle bug in the convergence check. Identify it and explain what incorrect behavior it could cause.

The bug is in the condition `loglik_history[-1] - loglik_history[-2] < tol`. Because it lacks an absolute value check (or a check ensuring the difference is positive), any *decrease* in log-likelihood will result in a negative number, which is strictly less than `tol`. While EM guarantees monotonic increases mathematically, numerical instability or precision issues can cause slight decreases. If this happens, or if the algorithm diverges severely, the loop will silently and prematurely break, falsely reporting convergence.

(b) Suppose this function is called with `sigma1_init=0.001` on a dataset where one observation happens to be very close to `mu1_init`. Explain what numerical issue could arise during the E-step and how you would fix it.

With a tiny σ_1 , the Gaussian PDF value for a point precisely at μ_1 will spike to an enormously high value. This can cause a floating-point overflow, resulting in `d1` becoming `inf`. Calculating `gamma` then results in `inf / inf`, which evaluates to `NaN` (Not a Number), breaking the entire M-step. Additionally, for points far from both means, `d1` and `d2` can both underflow to exactly `0.0`, causing a `0 / 0` division error. **Fix:** Perform the E-step computations in log-space using the **log-sum-exp trick** to compute `gamma` robustly. Also, add a small constant penalty (e.g., `1e-6`) to the variance updates in the M-step to prevent variances from collapsing to zero.

(c) Does this implementation handle the label-switching symmetry of the mixture model? That is, if you swap the roles of component 1 and component 2 in the initialization, do you get an equivalent solution? Explain why or why not.

Yes, it handles label-switching naturally. The EM algorithm is entirely symmetric. If you swap the initial parameters (replacing π with $1 - \pi$, μ_1 with μ_2 , and σ_1 with σ_2), the computed arrays for `d1` and `d2` will exactly swap. Consequently, `gamma` becomes `1 - gamma`. The

M-step will then update the swapped components symmetrically, returning an identical log-likelihood history and an equivalent final mixture model (just with the arbitrary labels reversed).

✓ Problem 3: EM for Missing Data

Consider a dataset of $n = 200$ paired measurements (X_i, Y_i) from a bivariate normal distribution, where 25% of Y values are missing at random.

(a) Write a function that implements the EM algorithm for estimating the bivariate normal parameters $(\mu_X, \mu_Y, \sigma_X^2, \sigma_Y^2, \rho)$ when Y has missing values. Use the following signature:

```
import numpy as np

def em_bivariate(X, Y, observed, max_iter=200, tol=1e-8):
    """EM for bivariate normal with missing Y values.

    Parameters
    -----
    X : np.ndarray
        Fully observed variable, shape (n,).
    Y : np.ndarray
        Partially observed variable, shape (n,). NaN for missing.
    observed : np.ndarray
        Boolean array, True where Y is observed.
    max_iter : int
        Maximum iterations.
    tol : float
        Convergence tolerance on max parameter change.

    Returns
    -----
    dict with keys: mu_x, mu_y, var_x, var_y, rho, n_iter
    """
    n = len(X)
    X_obs = X[observed]
    Y_obs = Y[observed]
    X_mis = X[~observed]

    # Initialize using complete cases
    mu_x = np.mean(X)
    var_x = np.var(X) # MLE variance (ddof=0)

    mu_y = np.mean(Y_obs)
    var_y = np.var(Y_obs)
    cov_xy = np.cov(X_obs, Y_obs, ddof=0)[0, 1]
    rho = cov_xy / np.sqrt(var_x * var_y)

    for iteration in range(max_iter):
        old_params = np.array([mu_y, var_y, rho])

        # E-step: Calculate expected values for missing data
        beta = rho * np.sqrt(var_y / var_x)
        y_mis_hat = mu_y + beta * (X_mis - mu_x)
        var_y_given_x = var_y * (1 - rho**2)

        # Compute expected sufficient statistics
        sum_y = np.sum(Y_obs) + np.sum(y_mis_hat)
        sum_y2 = np.sum(Y_obs**2) + np.sum(y_mis_hat**2 + var_y_given_x)
        sum_xy = np.sum(X_obs * Y_obs) + np.sum(X_mis * y_mis_hat)

        # M-step: Update parameters using expected sufficient statistics
        mu_y = sum_y / n
        var_y = sum_y2 / n - mu_y**2
        cov_xy = sum_xy / n - mu_x * mu_y
        rho = cov_xy / np.sqrt(var_x * var_y)

        # Check convergence
        current_params = np.array([mu_y, var_y, rho])
        if np.max(np.abs(current_params - old_params)) < tol:
            break

    return {
        'mu_x': mu_x,
        'mu_y': mu_y,
        'var_x': var_x,
```

```

        'var_y': var_y,
        'rho': rho,
        'n_iter': iteration + 1
    }

```

(b) Test your implementation on the following simulated data. Compare the EM estimates with the complete-case estimates (using only observations where Y is observed) and the full-data estimates (using the true Y values before deletion).

```

np.random.seed(42)
n = 200
mu = np.array([3.0, 7.0])
rho_true = 0.6
sx, sy = 1.5, 2.0
Sigma = np.array([[sx**2, rho_true * sx * sy],
                  [rho_true * sx * sy, sy**2]])

data = np.random.multivariate_normal(mu, Sigma, n)
X, Y_full = data[:, 0], data[:, 1]

# Introduce 25% MAR missingness
prob_miss = 0.25 * np.ones(n)
missing = np.random.binomial(1, prob_miss, n).astype(bool)
Y = Y_full.copy()
Y[missing] = np.nan
observed = ~missing

result = em_bivariate(X, Y, observed, max_iter=200, tol=1e-8)
print(result)

{'mu_x': np.float64(2.9575587837375408), 'mu_y': np.float64(7.021104128537144), 'var_x': np.float64(2.008791570670503)

```

(c) Which estimator do you expect to have smaller variance for ρ , the EM estimator or the complete-case estimator? Explain why.

The EM estimator is expected to have a smaller variance for ρ .

Why: The complete-case estimator entirely discards any pair where Y is missing, effectively throwing away valid data and reducing your overall sample size. In contrast, the EM algorithm uses all available information, including the observed X values from the incomplete pairs. Because X and Y are jointly distributed, the isolated X observations still carry statistical information about the joint distribution and the missing Y values. By leveraging this partial data rather than discarding it, the EM algorithm increases statistical efficiency, resulting in a smaller variance for the parameter estimates.

✓ Problem 4: Zero-Inflated Poisson Model Selection

The following code fits a standard Poisson model and a zero-inflated Poisson (ZIP) model to a dataset. Read the code and answer the questions.

```

import numpy as np
from scipy import stats

def fit_poisson(y):
    """Fit a standard Poisson model by MLE."""
    lam_hat = np.mean(y)
    ll = np.sum(stats.poisson.logpmf(y, lam_hat))
    return {"lam": lam_hat, "loglik": ll, "n_params": 1}

def fit_zip_em(y, max_iter=200, tol=1e-8):
    """Fit a zero-inflated Poisson model by EM."""
    n = len(y)
    pi = 0.3
    lam = np.mean(y[y > 0])
    is_zero = (y == 0)
    loglik_history = []

    for iteration in range(max_iter):
        gamma = np.zeros(n)
        gamma[is_zero] = pi / (pi + (1 - pi) * np.exp(-lam))

        ll = (np.sum(is_zero) * np.log(pi + (1 - pi) * np.exp(-lam))
              + np.sum(~is_zero * (np.log(1 - pi)
                                   + stats.poisson.logpmf(y, lam))))

        loglik_history.append(ll)

```

```

        if iteration > 0 and abs(loglik_history[-1] - loglik_history[-2]) < tol:
            break

    pi = np.mean(gamma)
    lam = np.sum((1 - gamma) * y) / np.sum(1 - gamma)

    return {"pi": pi, "lam": lam, "loglik": ll,
            "n_params": 2, "loglik_history": loglik_history}

```

(a) In the E-step, why is $\gamma_i = 0$ for all observations with $y_i > 0$? Explain using the structure of the ZIP model.

In a Zero-Inflated Poisson (ZIP) model, the data is generated from either a point mass at zero (with probability π) or a standard Poisson distribution (with probability $1 - \pi$). The point mass component can *only* produce zeros. Therefore, if an observation $y_i > 0$, it is impossible for it to have come from the zero-inflation component. Its posterior probability (responsibility γ_i) of belonging to the zero state is exactly 0.

(b) Given that the Poisson model is nested within the ZIP model (setting $\pi = 0$ recovers the Poisson), a natural model comparison tool is the likelihood ratio test. However, the standard chi-squared approximation for the LRT is not valid here. Explain why, in terms of the parameter space and the null hypothesis.

The null hypothesis that reduces the ZIP model to a standard Poisson model is $H_0 : \pi = 0$. Because π is a probability ($\pi \in [0, 1]$), this null value lies exactly on the **boundary of the parameter space**. The standard chi-squared approximation for the Likelihood Ratio Test (Wilks' theorem) strictly requires the null parameter to reside in the *interior* of the parameter space. When testing on a boundary, the test statistic follows a non-standard asymptotic distribution (typically a mixture of chi-squared distributions).

(c) Using the AIC ($AIC = -2\ell + 2k$, where k is the number of parameters), compare the Poisson and ZIP fits on the following data. Which model does AIC prefer?

We calculate the AIC for both models using the formula $AIC = -2\ell + 2k$:

- **Poisson Model:**

- Log-likelihood (ℓ): -496.09
- Parameters (k): 1 (λ)
- $AIC_{\text{Poisson}}: -2(-496.09) + 2(1) \approx \mathbf{994.18}$

- **ZIP Model:**

- Log-likelihood (ℓ): -483.76
- Parameters (k): 2 (π, λ)
- $AIC_{\text{ZIP}}: -2(-483.76) + 2(2) \approx \mathbf{971.53}$

Conclusion: The AIC for the ZIP model (971.53) is lower than the AIC for the standard Poisson model (994.18). Therefore, **AIC prefers the Zero-Inflated Poisson (ZIP) model**, which correctly reflects the underlying data-generating process.

```

np.random.seed(10)
n = 300
pi_true = 0.2
lam_true = 2.0
z = np.random.binomial(1, pi_true, n)
y = np.where(z == 1, 0, np.random.poisson(lam_true, n))

poisson_fit = fit_poisson(y)
zip_fit = fit_zip_em(y)

print("Poisson fit:", poisson_fit)
print("ZIP fit:", zip_fit)

```

```

Poisson fit: {'lam': np.float64(1.52), 'loglik': np.float64(-496.0909991283401), 'n_params': 1}
ZIP fit: {'pi': np.float64(0.182777331600003), 'lam': np.float64(1.859958195941797), 'loglik': np.float64(-483.762731

```

✓ Problem 5: EM with Asymmetric Components

Consider a two-component mixture where component 1 is Exponential(λ) (supported on $[0, \infty)$) and component 2 is $N(\mu, \sigma^2)$ (supported on $(-\infty, \infty)$):

$$f(y_i|\theta) = \pi \lambda e^{-\lambda y_i} \mathbb{1}(y_i > 0) + (1 - \pi) \phi(y_i|\mu, \sigma^2)$$

(a) Derive the E-step and M-step for this model. Pay careful attention to how the exponential component's support restriction affects the responsibility computation for observations with $y_i \leq 0$.

E-step: Responsibilities

The responsibility $\gamma_i^{(t)} = P(z_i = 1 | y_i, \theta^{(t)})$ is the posterior probability that y_i belongs to the Exponential component. Because the Exponential distribution has zero probability for $y_i \leq 0$, the indicator function $\mathbb{1}(y_i > 0)$ acts as a strict gate:

- If $y_i \leq 0$: The observation cannot possibly come from the Exponential component.

$$\gamma_i^{(t)} = 0$$

- If $y_i > 0$: We use Bayes' theorem with both densities.

$$\gamma_i^{(t)} = \frac{\pi^{(t)} \lambda^{(t)} e^{-\lambda^{(t)} y_i}}{\pi^{(t)} \lambda^{(t)} e^{-\lambda^{(t)} y_i} + (1 - \pi^{(t)}) \phi(y_i | \mu^{(t)}, (\sigma^2)^{(t)})}$$

where $\phi(y_i | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mu)^2}{2\sigma^2}\right)$. For component 2 (Normal), the responsibility is simply $1 - \gamma_i^{(t)}$.

M-step: Parameter Updates

We maximize the expected complete-data log-likelihood. Because $\gamma_i^{(t)} = 0$ for all $y_i \leq 0$, the Exponential parameter λ is only updated using positive observations.

1. Update for π :

$$\pi^{(t+1)} = \frac{1}{n} \sum_{i=1}^n \gamma_i^{(t)}$$

2. Update for λ :

$$\lambda^{(t+1)} = \frac{\sum_{i=1}^n \gamma_i^{(t)}}{\sum_{i=1}^n \gamma_i^{(t)} y_i}$$

(Note: Since $\gamma_i^{(t)} = 0$ when $y_i \leq 0$, negative values of y_i appropriately do not contribute to the rate parameter estimate).

3. Update for μ :

$$\mu^{(t+1)} = \frac{\sum_{i=1}^n (1 - \gamma_i^{(t)}) y_i}{\sum_{i=1}^n (1 - \gamma_i^{(t)})}$$

4. Update for σ^2 :

$$(\sigma^2)^{(t+1)} = \frac{\sum_{i=1}^n (1 - \gamma_i^{(t)}) (y_i - \mu^{(t+1)})^2}{\sum_{i=1}^n (1 - \gamma_i^{(t)})}$$

(b) Implement the EM algorithm for this model:

```
import numpy as np
from scipy import stats

def em_exp_normal_mixture(y, pi_init, lam_init, mu_init, sigma_init,
                           max_iter=200, tol=1e-8):
    """EM for exponential-normal mixture.

    Parameters
    -----
    y : np.ndarray
        Observed data of shape (n,).
    pi_init : float
        Initial mixing proportion for the exponential component.
    lam_init : float
        Initial rate for the exponential component.
    mu_init : float
        Initial mean for the normal component.
    sigma_init : float
        Initial std dev for the normal component.
    max_iter : int
        Maximum iterations.
    tol : float
        Convergence tolerance on log-likelihood change.

    Returns
    -----
    dict with keys: pi, lam, mu, sigma, loglik_history
    """
    n = len(y)
```

```

.. .. ..
pi = pi_init
lam = lam_init
mu = mu_init
sigma = sigma_init
loglik_history = []

# Pre-compute a mask for positive values
pos_mask = (y > 0)

for _ in range(max_iter):
    # E-step
    pdf_exp = np.zeros(n)
    pdf_exp[pos_mask] = lam * np.exp(-lam * y[pos_mask])
    pdf_norm = stats.norm.pdf(y, mu, sigma)

    marginal = pi * pdf_exp + (1 - pi) * pdf_norm

    # Log-likelihood
    ll = np.sum(np.log(marginal))

    # Convergence check
    if len(loglik_history) > 0 and abs(ll - loglik_history[-1]) < tol:
        loglik_history.append(ll)
        break
    loglik_history.append(ll)

    # Responsibilities
    gamma = (pi * pdf_exp) / marginal

    # M-step
    sum_gamma = np.sum(gamma)
    sum_comp2 = np.sum(1 - gamma)

    pi = sum_gamma / n
    lam = sum_gamma / np.sum(gamma * y)
    mu = np.sum((1 - gamma) * y) / sum_comp2
    sigma = np.sqrt(np.sum((1 - gamma) * (y - mu)**2) / sum_comp2)

return {
    'pi': pi,
    'lam': lam,
    'mu': mu,
    'sigma': sigma,
    'loglik_history': loglik_history
}

```

(c) Test on the following data and report your estimates:

```

np.random.seed(42)
n = 400
pi_true = 0.4
lam_true = 2.0
mu_true, sigma_true = -1.0, 1.5
z = np.random.binomial(1, pi_true, n)
y = np.where(z == 1,
             np.random.exponential(1 / lam_true, n),
             np.random.normal(mu_true, sigma_true, n))

result = em_exp_normal_mixture(y, pi_init=0.5, lam_init=1.0,
                               mu_init=0.0, sigma_init=2.0)
print(result)

{'pi': np.float64(0.42235452741743484), 'lam': np.float64(1.8425779897336965), 'mu': np.float64(-0.9123486611501683),

```


